

Resumo Teórico — Prova de C

Structs, Vetores de Structs e Modularização — baseado nos 10 exercícios resolvidos

Este material reúne a teoria essencial por trás dos 10 exercícios resolvidos (cadastros de clientes, produtos, alunos, livros, carros, contas, imóveis, jogadores e funcionários). Todos seguem o mesmo padrão: **struct** + **vetor de structs** + **funções modularizadas** + **menu com laço**.

1. Structs (Estruturas)

Uma **struct** agrupa variáveis de tipos diferentes sob um único nome, representando uma "entidade" do mundo real (um Cliente, um Produto, um Aluno...).

```
typedef struct {
    int codigo;
    int idade;
    char telefone[20];
} Cliente;
```

Pontos importantes

- O **typedef** permite usar *Cliente* diretamente como tipo, sem escrever *struct Cliente* toda vez.
- Cada campo é acessado com o operador ponto: **variavel.campo** (ex: *cliente.idade*).
- Quando a struct é acessada por **ponteiro**, usa-se a seta: **ponteiro->campo**.
- Strings dentro de struct são vetores de **char** com tamanho fixo (ex: *char telefone[20]*), nunca *char** sem alocação.
- Structs podem ser copiadas inteiras com **=** (ex: *a = b;*), diferente de vetores.

✓ **Pode: struct com campos de tipos diferentes (int, float, char[]).**

✗ **Não pode: comparar structs inteiras com == — isso não compila em C (é preciso comparar campo a campo).**

2. Vetores de Structs

Como o número de cadastros não é fixo, usamos um **vetor com tamanho máximo** (ex: *#define MAX 100*) e uma variável **total** (ou *contador*) que guarda quantas posições já foram realmente preenchidas.

```
#define MAX_CLIENTES 100
Cliente clientes[MAX_CLIENTES];
int total = 0; // quantos já foram cadastrados
```

Por que precisamos da variável 'total'?

O vetor sempre tem tamanho MAX (ex: 100), mas isso não significa que todas as posições estão "em uso". A variável *total* indica o limite real de dados válidos. Percorrer o vetor sempre é feito como:

```
for (int i = 0; i < total; i++) {
    // usa vetor[i]
}
```

Atenção: Percorrer até *MAX* em vez de até *total* é um erro comum: acessaria posições "vazias" (lixo de memória), gerando resultados incorretos ou comportamento indefinido.

Índice do vetor

O primeiro elemento é sempre **índice 0**, e o último elemento válido é **total - 1** (como no exercício 7, em que o usuário escolhe uma posição para depositar — é obrigatório validar se $0 \leq \text{índice} < \text{total}$ antes de usar).

3. Modularização (Funções)

Modularizar significa dividir o programa em funções pequenas, cada uma com uma responsabilidade única (cadastrar, listar, buscar, calcular...), em vez de colocar tudo dentro do *main*.

Passagem de vetor de structs para função

Vetores em C são passados **por referência automaticamente** (na verdade, o que é passado é o endereço do primeiro elemento). Ou seja, qualquer alteração feita dentro da função reflete no vetor original, **sem precisar de ponteiro explícito**.

```
void listarClientes(Cliente clientes[], int total) {
    for (int i = 0; i < total; i++) {
        printf("%d\n", clientes[i].codigo);
    }
}
```

E o 'total'? Por que a função devolve (return) o novo total?

Diferente do vetor, uma variável simples como *int total* é passada **por valor** — a função recebe uma cópia. Se a função apenas alterar essa cópia, o *total* do *main* não muda. Por isso, nas funções de cadastro usamos:

```
int cadastrarCliente(Cliente clientes[], int total) {
    // ...cadastra na posição 'total'...
    total++;
    return total;
}

// no main:
total = cadastrarCliente(clientes, total);
```

Atenção: Alternativa possível (não usada aqui, mas cabe em prova teórica): passar *int *total* por ponteiro e alterar com *(*total)++* dentro da função, dispensando o *return*.

✓ **Pode:** passar o vetor e o total como parâmetros e devolver o total atualizado.

✗ **Não pode:** esperar que alterar 'total' dentro da função mude automaticamente a variável do main, sem retorno ou ponteiro.

4. Estrutura de Menu (do-while + switch)

Todos os exercícios usam o mesmo esqueleto: um laço **do-while** que repete o menu até o usuário escolher "Sair", e um **switch** para decidir qual função chamar.

```
int opcao;
do {
```

```
printf("1. Cadastrar\n2. Listar\n3. Sair\n");
scanf("%d", &opcao);

switch (opcao) {
    case 1: /* ... */ break;
    case 2: /* ... */ break;
    case 3: printf("Saindo...\n"); break;
    default: printf("Opcao invalida\n");
}
} while (opcao != 3);
```

- **do-while** é usado (em vez de *while*) porque o menu precisa aparecer **pelo menos uma vez**, antes de qualquer verificação.
- O **break** dentro de cada case é obrigatório — sem ele, a execução "cai" para o próximo case (fall-through).
- O **default** trata opções inválidas digitadas pelo usuário.

5. Entrada de Dados com scanf

Especificadores mais usados

Tipo	Especificador	Exemplo
int	%d	scanf("%d", &codigo);
float	%f	scanf("%f", &preco);
string (char[])	%s ou %[^\n]	scanf(" %19[^\n]", telefone);

Atenção: Sempre usar **&** antes de variáveis simples (int, float) no scanf — exceto para vetores/strings, que já são endereços por natureza (*char telefone[20]* não usa &).

✗ Não pode: usar %s para ler strings com espaço ("João da Silva") — ele para no primeiro espaço. Por isso usamos %[^\n].

6. Lógicas Recorrentes nos Exercícios

6.1 Soma / Total acumulado (Ex. 2 — estoque)

```
float total = 0;
for (int i = 0; i < total_itens; i++) {
    total += produtos[i].quantidade * produtos[i].preco;
}
```

Padrão do **acumulador**: variável iniciada em 0, somada dentro do laço.

6.2 Média (Ex. 3 — notas)

```
float media = (aluno.nota1 + aluno.nota2) / 2;
```

Para **média geral de vários registros** (como no exercício 8), soma-se tudo e divide-se pelo *total*:

```
float soma = 0;
for (int i = 0; i < total; i++) soma += imoveis[i].consumoKwh;
float media = soma / total;
```

6.3 Filtro condicional (Ex. 4, 5, 8, 10)

Percorre-se o vetor inteiro e imprime-se apenas os elementos que satisfazem uma condição com **if**. Pode ser uma condição simples (ano > 2020) ou **dupla**, combinando com **&&** (E lógico):

```
if (funcionarios[i].idade > 40 && funcionarios[i].salario > 5000.0) {
    contador++;
}
```

- **&&** (E) — as duas condições precisam ser verdadeiras.
- **||** (OU) — basta uma condição ser verdadeira.
- **!** (NÃO) — inverte o valor lógico da condição.

6.4 Busca do maior/menor valor (Ex. 6)

```
int indiceMaior = 0;
for (int i = 1; i < total; i++) {
    if (alunos[i].altura > alunos[indiceMaior].altura) {
        indiceMaior = i;
    }
}
```

Ideia central: **guardar o índice** (não o valor) do maior elemento encontrado até o momento, começando a comparação em $i = 1$ (já que o índice 0 é o "maior" inicial).

6.5 Busca por ID/código específico (Ex. 9)

```
for (int i = 0; i < total; i++) {
    if (jogadores[i].idJogador == id) {
        printf("Encontrado!");
        return; // encerra a busca
    }
}
printf("Nao encontrado.");
```

Atenção: O *return* (ou uma flag/booleana) dentro do laço é o que evita continuar procurando depois de já ter achado o item.

6.6 Alteração de valor existente (Ex. 7)

```
contas[indice].saldo += valor; // soma ao saldo já existente</br>
```

Diferente do cadastro (que cria um novo elemento), aqui **acessamos um índice já existente** do vetor e atualizamos um dos seus campos.

7. Erros Comuns que Costumam Cair em Prova

- Esquecer o **&** no scanf de variáveis simples (int, float).
- Confundir **total** (quantidade preenchida) com **MAX** (capacidade do vetor).
- Esquecer o **break** dentro do switch.
- Comparar structs inteiras com **==** (não é permitido em C).

- Não validar o **índice** antes de acessar uma posição do vetor (Ex. 7).
- Esquecer de **devolver/retornar** o total atualizado após cadastrar (ele é passado por valor).
- Iniciar o acumulador de soma com um valor diferente de 0.
- Trocar **&&** por **||** em filtros com duas condições (isso muda completamente o resultado — ex. 10 pede E, não OU).

8. Tabela-Resumo dos 10 Exercícios

#	Struct	Operação-chave
1	Cliente	Cadastro + Listagem simples
2	Produto	Soma acumulada (qtd × preço)
3	Aluno	Cálculo de média (2 notas)
4	Livro	Filtro simples (ano > 2020)
5	Carro	Filtro com valor digitado pelo usuário
6	Perfil	Busca do maior valor (altura)
7	Conta	Alteração de valor por índice
8	Imóvel	Média geral + filtro acima da média
9	Jogador	Busca por ID (varredura)
10	Funcionario	Filtro duplo (idade E salário) + contador

9. Checklist Rápido Antes da Prova

- Sei declarar uma **struct** com *typedef* e acessar campos com ponto (.).
- Sei declarar um **vetor de structs** com tamanho máximo (*#define*) e controlar com uma variável *total*.
- Sei escrever uma função de **cadastro** que recebe o vetor e o total, e devolve o novo total.
- Sei escrever um **laço for** que percorre só até *total* (não até MAX).
- Sei diferenciar **filtro** (if dentro do for) de **busca** (if + return/break ao achar).
- Sei calcular **soma**, **média** e encontrar o **maior/menor** valor de um vetor.
- Sei montar o **menu** com do-while + switch + break.
- Sei usar **&&** para condições duplas (e não confundir com ||).